MODULE -1

Introduction To Python

What is programming ?

Programming is a way to instruct the computer to perform various tasks.

History of python

* Python is widely used general-purpose high level programming language.
* It was initially designed by Guido Van Rossum in 1991. and developed by python software foundation.
* It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code.
* Python 2.0 was released on 16 october 2000.
* Python 3.0 was released on 3 December 2008.
* Latest version 3.12.

The language's core philosophy

* It has an orthogonal, explicit and minimalist design that makes code easy to understand and easy to predict.
* Python approaches complexity by providing a simple core language and splitting applications-specific tools into a large set of modular library components.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Python variables: A variable is the name given to a memory location in a program.
Variable is like a container use to store a value.
for example:-

```
a = 30
b = "ABC"
C = 71.22
```

Keywords: Keywords are the reserved words in python.
E.g. import, break, if, etc.

Identifiers: Identifier is a class / function / variable name. module or other object..

Rules For defining a Variable name

→ A variable name can contain alphabets, digits, and underscores.

→ A variable name can only start with an alphabet and underscore.

→ A variable name can't start with a digit.

→ No white space is allowed to be used inside a variable name.

Examples of few valid variable names are-

abc, two3, _code, three, etc.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Datatypes:

Python datatypes are used to define the type of a variable. It defines what type of data we are going to store in a variable.

The data types are:-

(1.) Numeric datatypes → Integers, Float, complex numbers

(2.) Sequence datatypes → strings, list, Tuple

(3.) Boolean

(4.) Dictionary

(5.) None (6.) Set

Note:

type() function is used to determine the type of python datatype.

Python is a fantastic language that automatically identifies the type of data for us -

Example:

a = 271 ⟹ identifies 'a' as a class <int>

b = 88.45 ⟹ identifies 'b' as class <float>

name = "xyz" ⟹ identifies 'name' as class <str>

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures
Download App Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Numeric Data

→ Python, numeric data type is used to hold numeric values.
→ Integers, floating - point numbers and complex numbers.
→ They are defined as int, float and complex classes in python.
→ int - holds signed integers of non- limited length.
→ float - holds floating decimal points and its accurate upto 15 decimal points.
→ complex - holds complex numbers

```
num1 = 5
print(num1, " is type", type(num1))
num2 = 2.0
print (num2, "is of type", type(num2))
num3 = 1+ 2j
print (num3, 'is of type', type (num3))
```

output:

```
5 is type <class 'int'>
2.0 is of type <class 'float'>
(1+2j) is of type <class' complex'>
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Sequencing Type

String

String is a sequence of characters enclosed in quotes. A string can be defined in python using single, double or triple Quotes.

```
str1 = "hi"
str2 = "bye"
str3 = "Hello"
print(str1, " is of type ", type(str1))
print(str2;" is of type ", type(str2))
print (str3," is of type ", type (str3))
```

output:

```
h is of type <class 'str'>
bye is of type <class 'str'>
hello is of type <class 'str'>
```

String functions:

Some of the mostly used functions to perform operations on or manipulate strings are:

1. len() function: This function returns the length of the string.

Eg. length of a:

```
a = "Gateway"
len(a) ⟶ return 7
```

Download Gateway Classes App
Helpline No.: 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App
Web Page

2. string.endswith("way") → This function tells whether the variable string ends with the string "way" or not. if string is "Gateway", it returns true for "way" since gateway ends with way.

3. string.count("c") → counts the total number of occurence of any character

4. string.capitalize() → This function capitalise the first character of a given string.

5. string.find(word) → This function finds a word and returns the index of first occurence of that word in the string.

(6.) string.replace(oldword, newword) → This function replaces the oldword with new word in the entire string.

Escape sequence characters
These are the sequence of characters after backslash '\'.

Escape sequence character comprises of more than one characters but represents one character when used within the strings.

Example: \n, \t, \', \\, etc.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures
Download App Web Page

List

List is an ordered collection of similar or different types of items separated by commas and enclosed within brackets [].

for example:

```
language = [" python", " c", "java"]
print (" the type of language ", type( language])
```

output

the type of language <class 'list'>

Tuple data type

Tuple is an ordered sequence of items same as a list. The only difference is that tuples are immutable. Tuples once created cannot be modified.

In python() , we use parenthesis () to store items for a tuple.

A tuple is an immutable data type in python.

Tuple methods (consider a typle a = (1, 7, 2)

1. a.count(1)
   a.count(1) will return numbers of times 1 occurs in a.
2. a.index(1): a.index(1) will return the index of first occurence of 1 in a.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

dictionary data type

dictionary is an ordered collection of items. It stores elements in key/ value pairs.
Here, keys are unique identifiers that are associated with each value.

Access dictionary values using keys

```
# create a dictionary named capital_city
capital_city = { 'Nepal ': 'Kathmandu ', 'Italy' : 'Rome' , 'England': 'London'}
print (capital_city ['Nepal']) # prints Kathmandu
print ( capital_city ['Kathmandu']) # throws error message.
```

```
d = {1:' gatewayclasses' , 2 :'python', 3: 'datatype'}
print (d)
print ( d.keys())
print ( d.values())
print ("1name is "+ d[1])
print ("3name "+ d[3])
```

output

```
dict_keys ( [1, 2, 3])
dict_values ( ['gatewayclasses', 'python', 'datatype']
1name is gatewayclasses
3name datatype
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Boolean

The boolean value can be of two types only i.e. either. True or False. The output <class 'bool'> indicates the variable is a Boolean data type.

```
a = True
type (a)
```

output

```
<class 'bool'>
```

Sets

Sets are used to store multiple items in a single variable.
It is one of 4 built-in data types in python.
A set is a collection which is unordered, unchangeable, unindexed.
Duplicate items are not allowed
You can remove items and add new items.

```
thisset = { "apple", "banana", "cherry"}
print (thisset )
```

output

```
{'apple', 'banana', 'cherry'}
```

```
thisset = { "apple" , "banana", "cherry", "apple"}
print ( thisset )
```

output

```
{'apple' , 'banana', 'cherry'}
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

input()

This function first takes the input from the user and converts it into a string. The type of the returned object always will be < class 'str' >. It does not evaluate the expression. - it just returns the complete statement as string.
For example,
Python provides a built-in function called input which takes the input from the user. When the input function is called it stops the program and waits for the user's input. When the user presses enter, the program resumes and returns what the user typed.

Example:

```
val = input ("Enter your value:")
print (val)
```

output

```
Enter your name xyz
  xyz
```

```
val = input ("Enter your value:\n")
print (val)
```

output

```
Enter your name
 xyz
xyz
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

How the input function works in python:

- When input() function executes program flow will be stopped until the user has given input.
- The text or message displayed on the output screen to ask user to enter an input value is optional i.e. prompt, which will be printed on the screen is optional.
- whatever you enter as input, the input function converts it into a string. If you enter an integer value still input() function converts it into a string. You need to explicitly convert it into an integer in your code using typecasting.

```
number = input ("enter the number:")
print (number)
```

Output

```
enter the number: 123
123
```

```
number = input ("enter the number\n")
print (number)
name = input ("enter the name\n")
print (name)
print (" type of number", type (number))
print ("type of name", type (name))
```

Output

```
enter the number
123
123
enter the name
Rani
Rani
type of number <class 'str'>
type of name <class 'str'>
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Raw_input()
This function works in older version (like python 2.x).
This function takes exactly what is typed from the keyboard, converts it to string, and then returns it to the variable in which we want to store it.

```
number= raw_input ("enter the number in decimal")
print ( number)
```

output

raw_input is not defined

Write a program to take number from user and add these number

```
x = int( input(" enter the first number"))
print (x)
y= int (input ("enter the second number"))
z = x+y
print(" the sum of two number ", z)
```

Write a program to find out the area of rectangle

```
x = int (input(" enter the length of rectangle\n"))
print (" the length of rectangle ", x)
y = int(input (" enter the width of rectangle\n"))
print(" the width of rectangle", y)
area = x * y
print (area)
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Gateway Classes

B.Tech - Second Year **Python Programming**

Write a program to find out the perimeter of rectangle.

```
x = int(input("enter the length of rectangle\n"))
print("the length of rectangle", x)
y = int(input("enter the width of rectangle\n"))
print("the width of rectangle", y)
peri = 2*(x+y)
print(Peri)
```

Write a program to find out area and perimeter of a square

```
a = float(input("enter the side of the square\n"))
print("the side of the square", a)
area = a*a
print("the area of the square", area)
perimeter = 4*a
print("the perimeter of square", perimeter)
```

Write a program to find out the area and circumference of circle

```
r = float(input("enter the radius of circle\n"))
print("the radius of circle", r)
area = 3.14 * r * r
perimeter = 2 * 3.14 * r
print("the area of circle", area)
print("the perimeter of circle", perimeter)
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Write a program to swap two number with using third variable

```
x = int( input (" enter the feirst number \n" ))
y = int (input (" enter second number \n" ))
print (" the value of x = ", x )
 print (" the value of y = ", y )
   z = x
   x = y
   y = z
   print (" the value of x after swapping ", x )
   print (" the value of y after swapping ", y )
```

Write a program to swap two number without using third variable

```
x = int( input (" enter the First number \n" ))
Y = int (input ( " enter second number \n" ))
print (" the value of x = ", x )
 print (" the value of y = ", y )
   x = x + y
   y = x - y
   x = x - y
   print (" the value of x after swapping ", x )
   print (" the value of y after swapping " y )
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Write a program to find out simple interest

```
p= float( input( "enter the principle "))
r= float( input(" enter the rate"))
t = float( input(" enter the time "))
si = (p * r * t )/100
print (" the simple interest ", si)
```

Write a program to find celcius to fahrenheit

```
celcius = int ( input(" Enter the temperature in
          celcius :\n"))
fahrenheit = (1.8 * celcius )+ 32
print(" Temperature in Fahrenheit :", fahrenheit)
```

Write a program to calculate average of two numbers

```
a = int( input(" enter first number" ))
print(" first number is", a)
b = int ( input ( " enter second number"))
print ( " second number is ", b)
avg= (a+b) /2
print (avg)
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures

operators in python

operators are used to perform operations on variables and value.

operand

An operand is a value that the operator needs to complete a task.

different types of operators are -

→ Airthmetic operators
→ Comparison operators
→ Assignment operators
→ Logical operators
→ Bitwise operators
→ Membership operators
→ special operators
→ Identity operators

Airthmetic operators

Airthmetic operators are used to execute mathematical functions . in python which include , addition , subtraction, multiplication, division, etc.

operator	Description	Example
(+) plus	Adds two operands	5+5 =10
(−) Minus	Right hand operand is Subtracted from the left-hand operand.	20-10 =10
(*) Multiplication	It multiplies the values on either side of the operands.	2 * 2 =4

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

operator	Description	Example
(/) Division	left hand operand divided by right-hand operand	50/5 = 10
(%) modlus	It divides the left hand operand by right-hand one and also returns a remainder.	7 % 2 = 1
(//) Floor division	Division that results in the whole number being adjusted to the left in the number line.	5.0 // 2 = 2.0
(**) Exponent	The left operand is raised to the power of the right.	10 to the power of 3 10 ** 3 = 1000

Floor Division: Floor division is a normal division operation, except it return the largest possible integer. The integer can either be less than the normal division output or equal to it.

The Floor division is an operation in python that allows us to divide two numbers and rounds the resultant value down to the nearest integer. The Floor division occurs through the (//) operator.

Floor function is signified by the ⌊⌋ symbol in mathematical terms.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Let us now understand the working of the floor division.

For example: [36/5]

1. performing the division first . we will divide 36 by 5

   $36 \div 5 = 7.2$

2. Now, we will perform the floor division on the obtained value i.e. 7.2

   $[7.2] = 7$

Let us now understand the working of all arithmetic operators via program or code.

```
a = 543
b = 32
x = 10
y = 20
```

output

```
print ( a//b )      ⟶   16
print ( a/b )       ⟶   16.97
print ( "x+y=" , x+y )   ⟶   x+y= 30
print ( " x-y=", x-y )   ⟶   x-y=-10
print ( "x * y=", x * y)   ⟶   x*y= 200
print ( " x%y =", x%y)   ⟶   x%y = 1
print ( " x**y =", x**y)   ⟶   x**y = 1000
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Python Comparison (Relational) operators

In python, the comparison operator is used to analyze either side of the values & decides the relation between them.

different types of comparison (relational) operators are -

(1.) == Equal
(2.) != Not equal
(3.) > greater than
(4.) < less than
(5.) >= Greater than or Equal to
(6.) <= Less than or Equal

(1.) Python (==) Equal comparison Operator

The equal operator is used to compare the two values and to give the result whether they are equal or not. If both the values are equal, then it gives the result as. True, if not False.

Example:

```
x =10
y= 3
print (" x==y", x==y)
a = 120
b= 120
print (" a==b",' a==b)
```

output

```
x==y False
a==b True
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

(2.) python (!=) Not Equal comparison operator

This Not equal operator is used to compare two operands and returns a True statement. if they are not equal. and False if they are equal.

Eg.
```
x=10
y=3
print("x!=y", x!=y)
a=120
b=120
print("a!=b", a!=b)
```

output

x!=y True

a!=b False

(3.) python (>) greater than comparison operator

This operator is mainly used in python to return True if the left-hand value is greater than the right one

Eg.
```
x=10
y=3
print("x>y", x>y)
```

output

x>y True

(4.) Python (<) less than comparison Operator

This operator compares the right-hand operator with the left-hand and if the right-hand one is greater than the left-hand one. it returns true statement.

Eg.
```
x=10
y=3
print("x<y", x<y)
```

output

x<y False

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

(5.) Python (>=) greater than equal to , (<=) less than equal to comparison operators

This operator compares the right-hand operator with the left-hand one and if the right-hand one is greater than the left-hand one, if returns True statement

Eg:-

```
x=10
y=3
print (x<=y)
print (x>=y)
```

output

False

True

Python Assignment Operators

The assignment operator is used to assign value to the event, property, or variable. we use this operator in python to assign values to variables. we have multiple Assignment operators. Let's list them down and know things better

= Assign value
+= Add AND
-= subtract AND
*= Multiply AND
/= Divide AND
%= Modulus AND
**= exponent AND
//= Floor Division

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Eg:-
```
a=10
print(a)
 a+=10
print ("a=", a)
 a*=10
print ("a=", a)
  b=5
  b/=2
  print ("b=", b)
  b//2
  print ("b=", b)
b%=2
print ("b=", b)
 b**=2
print ("b=", b)
  c=15
  c//=2
print ("c=", c)
```

Output

```
a= 20
a= 200
b= 2.5
b= 1.0
b= 1.0
b= 1.0
c= 7
```

Download App

Download Gateway Classes App
Helpline No. : 7455 9612 84

PDF Notes | PYQs Solution | Recorded Lectures | Web Page

Python Logical operators

Logical operators are used in any programming language to make decisions based on multiple conditions. In python, we use logical operators to determine whether a condition is true or False by taking operand values as a base

AND - logical AND
OR - logical OR
Not - logical NOT

X	Y	x and Y	x or y	not(x)	not(Y)
0(False)	0	0	0	1	1
0	1	0	1	1	0
1(true)	0	0	1	0	1
1	1	1	1	0	0

Eg:-

```
x = 1072
y = 2074
print ('x and y is', x and y)
print ('x or y is', x or y)
print ('not(x and y) is', not(x and y))
```

output

```
x and y is True
x or y is True
not(x and y) is False
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Python Bitwise operators

Bitwise operators are used in python to perform the operations on binary numerals or bit patterns. It operates bit by bit.

- & Binary AND
- | Binary OR
- ^ Binary XOR
- ~ Binary ones complement
- << Binary Left shift
- >> Binary Right shift

x	y	x&y	x|y	x^y	~(x)	~(y)
0(false)	0	0	0	0	1	1
0	1	0	1	1	1	0
1(true)	0	0	1	1	0	1
1	1	1	1	0	0	0

Bitwise AND operator

	2^7 128	2^6 64	2^5 32	2^4 16	2^3 8	2^2 4	2^1 2	2^0 1
10	0	0	0	0	1	0	1	0
4	0	0	0	0	0	1	0	0
10&4	0	0	0	0	0	0	0	0

Download App

Download Gateway Classes App
Helpline No. : 7455 9612 84

PDF Notes | PYQs Solution | Recorded Lectures | Web Page

```
a=10
b=4
print(a&b)
```

output

0

Bitwise OR operator

	2^7 128	2^6 64	2^5 32	2^4 16	2^3 8	2^2 4	2^1 2	2^0 1
10	0	0	0	0	1	0	1	0
4	0	0	0	0	0	1	0	0
10|4	0	0	0	0	1	1	1	0

Eg.

```
a=10
b=4
print(a|b)
```

output

14

Bitwise XOR operator

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
10	0	0	0	0	1	0	1	0
6	0	0	0	0	0	1	1	0
10^6	0	0	0	0	1	1	0	0

Eg.

```
a=10
b=6
print(a^b)
```

output

12

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

Bitwise ONE's complement operator

	2^7 128	2^6 64	2^5 32	2^4 16	2^3 8	2^2 4	2^1 2	2^0 1
10	0	0	0	0	1	0	1	0
~10	1	1	1	1	0	1	0	1

Eg:-

```
a=10
print(~a)
```

output

-11

Bitwise left shift operator

	2^7 128	2^6 64	2^5 32	2^4 16	2^3 8	2^2 4	2^1 2	2^0 1
10	0	0	0	0	1	0	1	0
10<<1	0	0	0	1	0	1	0	0 (extra bit added)

Eg:-

```
a=10
print(a<<1)
print(a<<2)
```

output

20

40

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes | PYQs Solution | Recorded Lectures | Web Page

Bitwise right shift operator

	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
	128	64	32	16	8	4	2	1
10	0	0	0	0	1	0	1	0
10771	0 (extra bit added)	0	0	0	0	1	0	1

Eg:

```
a=10
print(a>>1)
print(a>>2)
```

output

5

2

Python membership operators

Membership operators are mainly used in python to find whether a value or variable is present in a sequence (string, tuple, list, set, and dictionary) "In and "Not In" are the two membership operators available in python.

(1.) Python In Membership operator

This operator is used in python to evaluate if a particular value is there or not in a sequence. The In operator returns True if it is the specified element found in the list, otherwise. it returns false.

```
list = [1, 2, 3, 4, 5]
b=2
if b in list:
 print("present")
  else:
  print("not")
```

output

present

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures
Download App Web Page

(2.) Python Not in Membership operator

"Not in" operator, the name itself expresses the meaning that something is not inside. It goes through a particular sequence to find whether a value in the specified list or not. If it finds there is no such value, it returns True, otherwise False.

```
list = [1, 2, 3, 4, 5]
b = 2
if b not in list:
  print ("hi")
else:
print (" bye")
```

output

bye

Identity operators

Identity operators are used to compare the objects if both the objects are actually of the same data type and share the same memory location.

There are different identity operators such as

'is' operator - Evaluates to True if the variables on either side of the operator point to the same object and False otherwise.

```
x = 5
y = 5
print(x is y)
```

output

true

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

operators precedence & Associativity

Python operator Precedence

Precedence	operator sign	operator Name
Highest	**	Exponentiation
	+x, -x, ~x	Unary positive, unary negative, bitwise negation
	*, /, //, %	Multiplication, division, Floor division, modulus
	+, -	Addition, Subtraction
	<<, >>	Left-shift, right-shift
	&	Bitwise AND
	^	Bitwise XOR
	|	Bitwise OR
	==, !=, <, <=, >, >=, is, is not	comparison, identity
	not	Boolean NOT
	and	Boolean AND
Lowest	or	Boolean OR

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

operators Associativity

operators	Description	Associativity
()	Parentheses	Left to right
x[index], x[index:index]	Subscription, slicing	"
await x	Await expression	N/A
**	Exponentiation	Right to left
+x, -x, ~x	Positive, negative, bitwise NOT	"
*, @, /, //, %	Multiplication, matrix, division, Floor division, remainder	Left to Right
+, -	Addition and subtraction	"
<<, >>	Shifts	"
&	Bitwise AND	"
^	Bitwise XOR	"
|	Bitwise OR	"
in, not in, is, is not, <, <=, >=, !=, ==	comparisons, membership, tests, identity tests	"
not x	Boolean NOT	Right to left
and	Boolean AND	left to right
or	Boolean OR	"
if-else	Conditional expression	Right to left
lamda	Lamda expression	N/A
:=	Assignment expression (walrus operator)	Right to left

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

precedence of '+' & '*'

```
expr = 10 + 20 * 30
print (expr)
```

output

610

```
X = 87676
print(X)
```

output

False

Associativity of python operators

If an expression contains two or more operators with the same precedence then operator. Associativity is used to determine. It can either be left to right or from right to left.

eg:-

```
100/10 * 10
10 * 10
100
print(100/10*10)
# left associative
print (5-2+3)# left associative}
Print(5 - (2+3)) # () is higher precedence -,+
Print (2**3**2) # ** is right associativity
```

output

100.0

6

0

512

Non. Associative Operators

In python, most operators have associativity, which means they are evaluated from left to right or right to left when they have the same precedence. However, there are a few operators that are non- associative, meaning they cannot be chained together.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Type Conversions

The act of changing an object's data type is known as type conversion. The python interpreter automatically performs implicit type conversion.

Python prevents implicit type conversion from losing data The user converts the data types of objects using specified functions in explicit type conversion, sometimes referred to as type casting, when type casting, data loss could happen if the object is forced to conform to a particular data type.

Two types of type conversions are -

- Implicit type
- Explicit type (type-casting)

Implicit type conversion

In implicit type conversion of data types in python, the python interpreter automatically converts one data type to another without using any user involvement.

```
x=10
print (" X is type of ", type(x))
y = 10.6
print(" y is type of ", type(y))
z = x + y
print (z)
print [" z is of type", type(z))
```

output

```
X is type of < class 'int'>
Y is type of < class 'Float'>
20.6
2 is of type <class
'Float'>
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Explicit type conversion

In explicit type conversion in python, the data type is manually changed by the user as per their requirement. with explicit type conversion, there is a risk of data loss since we are forcing an expression to be changed in some specific data type.

Converting integer to Float

int(a, base): This function converts any datatype to an integer: 'Base' specifies the base in which the string is if the data type is a string.

float(): This function is used to convert any data type to a floating-point numbers.

```
s= "10010"
c = int(s,2)
print("after converting to integer base:", c)
e = float(c)
print("after converting to float",e)
f = float(s)
print("after converting to float from string",f)
```

Output

```
after converting to integer base: 18
after converting to float 18.0
after converting to float from string 10010.0
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

python type conversion using ord(), hex(), oct()

ord(): The function is used to convert a character to an integer

hex(): This function is used to convert an integer to a hexadecimal string.

oct() This function is to convert an integer to an octal string.

NOTE

Python does not have a character datatype, a single character is simply a string with a length of 1.

```
S = "4"
c = ord(s)
print(" after converting string to integer:", c) # ASCII
                                                  value of 4 is 52
c = hex(56)
print(" after converting integer to hexadecimal", c)
  c= oct(56)
  print(" after converting integer to octadecimal", c)
```

output

```
after converting string to integer 52
after converting integer to hexadecimal 0x38
after converting integer to octadecimal 0o70.
```


Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

Python code to demonstrate Type conversion using dict(), complex(), str()

dict() This function is used to convert a tuple of order (key, value) into a dictionary.

str(): Used to convert an integer into a string.

complex(real, imag):

This function converts real numbers to complex (real, img) number.

```
a=1
b=2
tup=(('a',1),('F',2),('g',3))
c= complex(1,2)
print("After converting integer to complex number:",c)
c= str(a)
print("After converting integer to string:",c)
c= dict(tup)
print("After converting tuple to dictionary:",c)
```

Output

```
After converting integer to complex number: (1+2j)
After converting integer to string:1
After converting tuple to dictionary:{'a':1,'F':2,'g':3}
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Python type conversion using tuple (), set(), list()

```
s = 'python'
c = tuple(s)
print(" after converting string to tuple ", c)
print( type(c))
c = set(s)
print(" after converting string to set ", c)
print (type(c))
c = list(s)
print(" after converting string to set", c)
print( type(c))
```

Output

after converting string to tuple ('p', 'y', 't', 'h', 'o', 'n')

<class 'tuple'>

after converting string to set {'h', 'p', 'y', 'n', 't', 'o'}

<class 'set'>

after converting string to set ['p', 'y', 't', 'h', 'o', 'n']

<class 'list'>

Convert ASCII value to characters

```
a = chr(76)
b = chr(77)
print (a)
print (b)
```

output

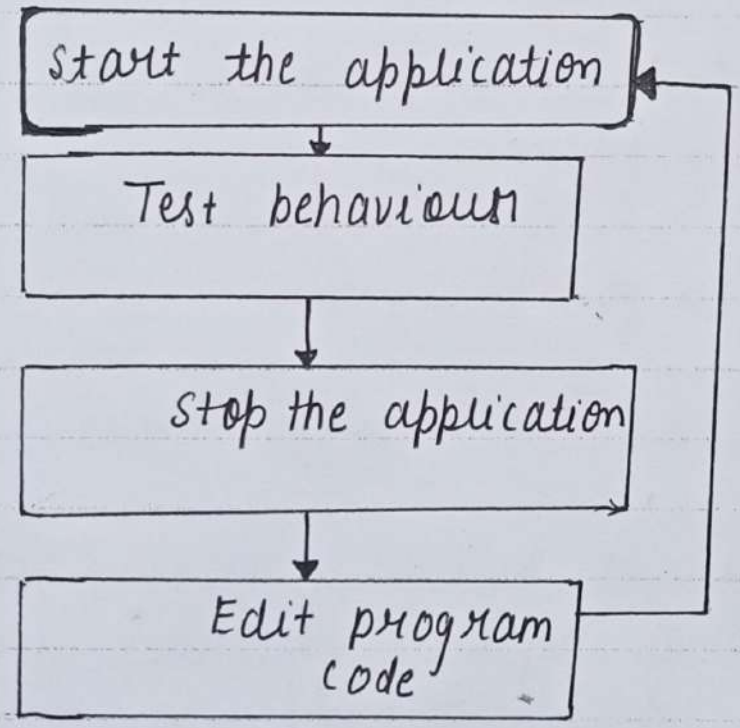
L

M

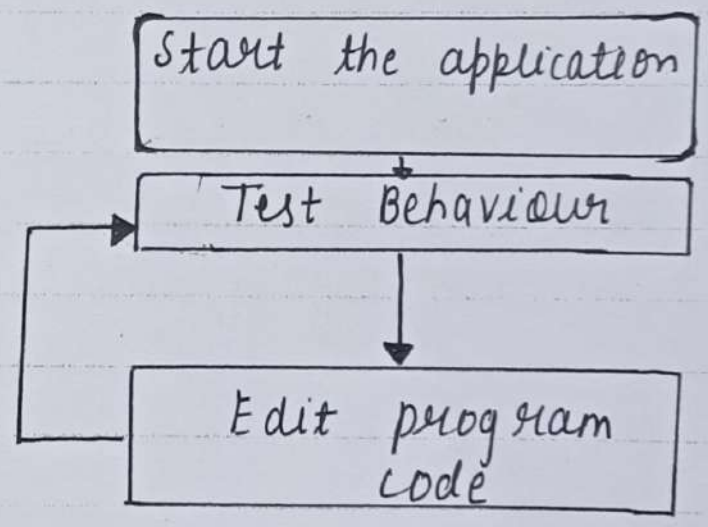
Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures
Download App Web Page

Programming cycle for python

Python's programming cycle is dramatically shorter than that of traditional programming cycle.
In python, there are no compile or link steps. Python programs simply import modules at runtime and use the objects they contain. Because of this, python programs run immediately after changes are made.
In cases where dynamic module reloding can be used, it is even possible to change and reload parts of a running program without stopping it at all.
pythons impact on the programming cycle is as Follows.



(a) Python's programming cycle



(b) Python's programming cycle with module reloding.

Since, python is interpreted, there is a rapid turnaround after program changes. And because python's phaser is embedded in python-based system. It is easy to modify programs at runtime.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

What is IDE

An integrated development environment (IDE) is a software applications that helps programmers develop software code efficiently. It increases developer productivity by combining capabilities such as software editing, building, testing, and packaging in an easy-to-use application. Just as writers use text editors and accountants use spreadsheets, software developers use IDEs to make their job easier.

Why IDE is important ?

code editing automation

Programming languages have rules for how statements must be structured. Because an IDE knows these rules. It contains many intelligent features for automatically writing or editing the source code.

Syntax highlighting

An IDE can format the written text by automatically making some words bold or italic, or by using different font colors. These visual cues make source code more readable and give instant feedback about accidental syntax errors.

Intelligent code completion

Various search items show up when you start typing words in a search engine. Similarly, an IDE can make suggestions to complete a code statement when the developer begins typing.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Refractoring support

Code refractoring is the process of restructuring the source code to make it more efficient and readable without changing its core functionality. IDEs can auto-refractor to some extent, allowing developers to improve their code quickly and easily. other team members understand readable code faster, which supports collaboration within the team.

Local build automation

IDEs increase programmer productivity by performing repeatable development tasks that are typically part of every code change.

The following are some examples of regular coding tasks that an IDE carries out.

Testing → The IDE allows developers to automate unit tests locally before the software is integrated with other developer's code and more complex integration tests are run.

Debugging → Debugging is the process of fixing any errors or bugs that testing reveals. One of the biggest values of an IDE for debugging purposes is that you can step through the code, line by line, as it runs and inspects code behaviour.

Some python IDE

→ Pycharm
→ Spyder
→ Pydev
→ IDLE
→ VISUAL STUDIO

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures
Download App Web Page

Python is considered as interpreted language

why python is interpreted language?

The most common saying, which is also published in various books, is that python is an interpreted language, however, the hidden fact is that python is both compiled and interpreted. This might seem contradictory at first, but it's important to understand how python's compilation and interpretation work together.

When you write a python program, it is initially in source code form. This source code needs to be transformed into an executable form that can be executed by the computer. There are two ways to do this:

Compilation: In this process, the source code is translated into machine code or bytecode ahead of time. The resulting compiled code can be executed directly without any further processing. Python uses a compiler to transform the source code into bytecode, which can be executed by the python virtual machine. This is how python code is executed in environments like pycharm, visual studio code, or other (IDEs) that use python's static analysis tools.

Interpretation: In this process, the source code is interpreted and executed directly by the interpreter at runtime. When you run a python script from the command line or an interactive shell, the interpreter reads the source code & executes it line by line. This is how python code is executed in interactive environments like Jupiter notebooks, or in web applications where python code is executed on the server side.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

Advantages of Interpreted language

Probability
Easy to learn and use
Rapid development
Flexibility
Debugging
interoperability

Disadvantages of Interpreted Language

Slower execution speed
Lack of static type checking
Security concerns

what is an Interpreted Language ?

Any programming language that isn't already in "machine code" before runtime is considered as interpreted language. An interpreted language is a computer programming language in which the instructions are executed without first being compiled into machine instructions.

In other words, unlike compiled languages, an interpreted language does not require prior translation, The translation occurs concurrently with the program's execution.

Instead of being instantly executed by the target computer, the instructions are read & executed by another program. Some interpretable scripting languages include Javascript, Perl, python, BASIC & others.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

what happen when you put semicolon at the end of python statement

The common meaning of a semicolon (;) in various programming languages is to end or discontinue the current statement. In programming languages such as C, C++, and Java, it's necessary to use a semicolon to end a line of code. However, this is not the case with python.

.A semicolon in python signifies separation rather than termination. It allows you to write multiple statements on a single line.

```
print ("hi"); print ("hello"); print ("bye")
```

Output

```
hi
hello
bye
```

In this example, we'll try to loop through two statements separated by a semicolon and see if the loop prints both or just the first one.

```
for i in range(4):
  print ("hi");
  print ("hello");
```

Output

```
Hi
Hello
Hi
Hello
Hi
Hello
Hi
Hello
```

Python will throw an error if you use a semicolon to separate a normal expression from a block statement i.e. loop.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

What type of language is python

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics developed by Guido Van Rossum. It was originally released in 1991. Designed to be easy as well as fun, the name "Python" is a nod to the British comedy group Monty Python.

There are some basic element of python

Datatype
operator
Variable
Function
Keywords

Tools to find bugs and perform static analysis

Writing efficient, reliable, and secure code is highly valued in software development.

With the popularity of python programming language skyrocketing in recent years, it has become increasingly important for developers to ensure their code is error-free secure, and optimized.

However, manually inspecting every lines of python code can be daunting and time-consuming. That is where static code analysis comes into play.

Static code analysis is the procedure of inspecting application source code without executing it.

It detects potential errors, security flaws.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Benefits of using static code Analysis for python

- identify bugs in early stages
- Enforces coding standards
- Returns Instant Feedbacks
- Reduce technical Debt

Static code analysis tools for python

→ pychecker and pylint are the static analysis tools that help to find bugs in python.

→ Pychecker is an opensource tool for static analysis that detects the bugs from source code and warns about the style and complexity of the bug.

→ Pylint is highly configurable and it acts like special programs to control warnings and errors. it is an extensive configuration File pylint also an opensource tool for static code analysis it looks for programming errors and is used for coding standards. It checks the length of each programming line. It checks the variable names according to the project style. It can also be used as a standalone program, it also integrates with python IDEs such as Pycharm, spyder, Eclipse & Jupyter

→ Pychecker can be simply installed by using pip package pip intall pychecker.

→ pip install pylint.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Features of Python

- **open source and free**

  It is an open-source language which means that anyone can download it, use it, and share it. Moreover, it is free of cost.

- **object-oriented**

  It supports object-oriented programming language features. For example, the concept of object and classes, encapsulation, inheritance, etc.

- **Extensible in Nature**

  It is extensible in nature which means that we can use python code in other languages. For example C, C++ also can compile that code in C or C++.

- **Integrated Language**

  We can easily integrate it with other languages such as C, C++, etc. Hence, it is an integrated language.

- **Interpreted Language**

  It uses an interpreter for converting the source code into machine code.

  This means that we execute the python code line by line. Hence, it becomes easy to debug the error & solve it.

- **Huge standard library**

  The libraries contain predefined modules and functions for certain tasks.

Download App

Download Gateway Classes App
Helpline No. : 7455 9612 84

PDF Notes | PYQs Solution | Recorded Lectures | Web Page

Boolean Expression

Boolean expressions are the expressions that evaluate a condition and result in a Boolean value i.e. true or false, Ex. (a>b and a>c) is a Boolean expression. It evaluates the condition by comparing. if 'a' is greater than 'b' and also if 'a' is greater than 'c'. If both the conditions are true only then does the Boolean expression result is true. If any one condition is not true then the Boolean expression will result in False.

Boolean expressions are written using Boolean operators (and), (or) and (not)

Example,

1. (x>1) and (x<5) - returns true if both the conditions are true, i.e. if the value of 'x' is between 1 and 5.

2. (x%x) or (x%1) - returns true if any one condition is true i.e. either 'x' is divisible by itself OR if 'x' is divisible by 1.

3. not (x==0) - returns true if the value of 'x' is other than 0. If the value of 'x' is 0 then it returns False.

Eg:-

```
x = 10
z = (x>1) and (x<5)
print (z)
```

output

False

```
x=10
y=12
print (x>22 or y<25)
```

True

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

PEP

PEP (Python enhancement proposal) is like a suggestion box for the Python programming language. It's way for people who work on Python to suggest new ideas or changes to Python , and it helps everyone discuss , plan, and decide if those ideas are good for python.
Just like when you have an idea, you write it down and share it with others. In the other Python community, PEPs are used to write down and share ideas for making python better. They explain what the idea is , why it's a good idea, and how it would work. Then, the python community talks about it, and if most people agree it's a good idea, it might become a part of python in the future.

In simple terms, a PEP is a way to suggest, discuss, and plan improvement or new features for Python , just like a suggestion box for your favourite programming language.

PEP stands for " Python Enhancement Proposal". PEPs are design documents that provide information to the python community or describe a new feature or improvement in the python programming language.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Gateway Classes

B.Tech - Second Year **Python Programming**

They are the primary mechanism for proposing and discussing changes to Python's core language and standard library.

The PEP process is used to suggest and debate ideas, document the design and implementation of new features, and in many cases, reach a consensus within the python community before a change is accepted and incorporated into python. PEPs cover a wide range of topics, from language enhancement to the development process itself.

PEPs are typically authored and submitted by python developers and enthusiasts and go through a review and approval process.
Some common types of PEPs include.

PEP 8
PEP 20
PEP 257
PEP 333, etc

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

PEP 8

PEP 8 is a python enhancement proposal that outlines the style guide for writing clean and readable python code. It provides a set of conventions and guidelines to make python code consistent and easy to understand. Following PEP 8 recommendations is considered good practice and helps ensure that python code is more accessible and maintainable.

Here are some key points from PEP 8 in plain language.

Indentation: Use 4 spaces for each level of indentation. Don't use tabs. Consistent indentation makes your code look neat and organised.

Maximum line length:
Limit each line of code to a maximum of 79 characters for code, and 72 character

Whitespace: Use a single space after commas, colons and semicolons, but not immediately before. Avoid extraneous whitespace (e.g. at the end of lines).

Comments: Write comments to explain complex or non-obvious parts of your code. Comments should be clear and concise. Use complete sentences and follow a consistent style for writing comments.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

Naming conventions: Use descriptive variable and function names. Variable names should be lowercase with words separated by underscore (e.g. my_variable_name).
Function names should be lowercase with words separated by underscores (e.g. my_function_name()).

Blank lines: Use blank lines to separate functions, classes, and blocks of code inside functions for better readability.

Whitespace in expressions & statements: Avoid extraneous whitespace within expressions and statements.
For example: don't use spaces immediately inside parenthesis or square brackets.

Encoding: Always specify the source code encoding as UTF-8 at the top of your python files to ensure proper character encoding.

Documentation strings (Docstrings): Write docstrings for modules, classes, and functions to explain their purpose & usage.

Imports: Import statements should usually be on separate lines and should come at the top of your file. Imports should be grouped in the following order: standard library imports, related third party imports, and local application / library specific imports.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes PYQs Solution Recorded Lectures Web Page

Gateway Classes

B.Tech - Second Year **Python Programming**

How memory is managed in python

Automatic Memory Allocation: when you create a new object in python (e.g. a variable), list, dictionary, etc) the memory for that object is allocated from the private heap. Python takes care of this memory allocation automatically.

Memory Deallocation : when the reference count of an object drops to zero (i.e. there are no more references to the object), Python's memory manager deallocates the memory associated with that object. The process is automatic and transparent to the programmer.

Garbage collection: Garbage collection is the process of removing the unused variables/ names and freeing the memory.
In python garbage collection is performed based on the reference count mechanism.
Garbage collector periodically checks and removes the objects from memory whose reference count is zero.

Memory Leaks: In some cases, if you create circular references or maintain references to objects unnecessarily, you can encounter memory leaks. These are situations where objects are not properly deallocated because their reference count never reaches zero. It's essential to be mindful of your code & ensure that you manage references appropriately to prevent memory leaks.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Print()

Python print() function prints the message to the screen or any other standard output device.

syntax: print (value(s), sep=' ', end='\n', file=file, flush=flush)

Parameters:

value(s): Any value, as many as you like. We will be converted to a string before printed.

sep='separator': (optional) specify how to separate the objects, if there is more than one. Default: ' '

end='end': (optional) specify what to print at the end. Default: '\n'.

File: (optional) An object with a write method. Default: sys.stdout

Flush: (optional) A Boolean, specifying if the output is flushed (True) or buffered (False). Default: False.

Return Type: It returns output to the screen.

How print() works in python?

You can pass variables, strings, numbers or other data types as one or more parameters when using the print() function. Then, these parameters are represented as strings by their respective str() functions. To create a single output string, the transformed strings are concatenated spaces between them.

Download Gateway Classes App
Helpline No.: 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page

```
name = "Alice"
age = 25
print ("Hello, my name is ", name, " and I am", age, "years old.")
```

Output:

Hello, my name is Alice and I am 25 years old.

python "end" parameter in print()

The end keyword is used to specify the content that is to be printed at the end of the execution of the print() function. By default, it is set o "\n"., which leads to the change of line after the execution of print() statement.

```
# This line will automatically add a new line before the # next print statement.
print("GATEWAY CLASSES")
# This print() function ends with "**" as set in the end argument
print ("HELLO GUYS", end="**")
print ("BYE"
  print (" how are you", end="$$")
```

output

GATEWAY CLASS
HELLO GUYS ** BYE
how are you $$

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

By default, print () separates the items by spaces. The optional sep = parameter sets a different separator. text.

```
print (12, 24, -2, sep=':')
print ('but', 'not', 'including', sep='**')
print ('but', 'not', 'including', sep='')
print ('hi', 'hello')
print("gateway.", "classes", sep='')
```

output

```
12:24:-2
but**not**including
butnotincluding
hi hello
gateway.classes
```

Flush parameter in python with print() function

The I/Os in python are generally buffered, meaning they are used in chunks. This is where flush comes in as it helps users to decide if they need the written content to be buffered or not. By default, it is set to false. If it is set to true, output will be written as a sequence of characters one after the other. This process is slow simply because it is easier to write in chunks rather than writing one character at a time. To understand the use case of the flush argument in the print() function.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Keyword

Python keywords are some predefined and reserved words in python that have special meanings. Keywords are used to define the syntax of the coding. The keyword cannot be used as an identifier, function, or variable name.
All the keywords in python are written in lowercase except True and False.

Rules for keywords in python

Python keywords cannot be used as identifiers.
All the keywords in python should be in lowercase except True and False.

```
import keyword
print(keyword.kwlist)
```

keyword.kwlist
This is a predefined variable of keyword module that stores all the keyword of python. Thus it can be used to display all the keyword of just by calling it.
python provides as in-built module keyword that allows you to know about the reserved keyword of python.
The keyword module allows you the functionality to know about the reserved words or keywords of python and to check whether the value of a variable is reserved word or not.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Gateway Classes

B.Tech - Second Year **Python Programming**

In case you are unaware of all the keywords of python you can use this module to retrieve this information. Also, it helps you to check whether a word is a keyword or not just by using its functions on python shell mode.

This function returns True if the parameter passed is a python keyword else returns False. The parameter can be a string or a variable storing a string. It accordingly compares the parameter with python keywords defined in the language and returns output.

```
import keyword
5 = "if"
t = "in"
u = "hello"
print(s, " is a keyword in python", keyword.iskeyword(s))
print(t," is a keyword in python", keyword.iskeyword(t))
print(u," is a keyword in python", keyword.iskeyword(u))
```

Output

```
if is a keyoword in python True
in is a keyword in python True
hello is a keyword in python False
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

Gateway Classes

B.Tech - Second Year **Python Programming**

IMPORT

Import in python is similar to #include header_file in C/C++. Python modules can get access to code from another module by importing the file/function using import. The import statement is the most common way of invoking the import machinery, but it is not the only way

```
import module_name
```

when the import is used, it searches the module initially in the local scope by calling __import__() function. The value returned by the function is then reflected in the output of the initial code.

```
import math
pie = math.pi
  print (" The value of pi is :", pie)
  The value of pi is :', 3.141592653589793
```

In the above code module, math is imported, and its variables can be accessed by considering it to be a class and pi as its object.
The value of pi is returned by __import__(). pi as a whole can be imported into our initial code, rather than importing the whole module.

```
from math import pi
  print(pi)
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

From module_name import *

In the below code module, math is not imported, rather just pi has been imported as a variable.

All the function and constants can be imported using *.

```
From math import *
print (pi)
  print ( Factorial (6))
    3.141592653589
    720
```

Identifiers

Python identifiers is the name we give to identify a variable, function, class, module or other subjects.

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Web Page
Download App

output questions

```
a=6
y=3
print(a/y)
  x=6
   y=3
    print(x//y)
x=5
 y=4
 print(x%y)
   x=5
    y=7
    print(x%y)
```

output

2.0

2

1

5

```
x=5
 y=4
 x+=y
   print(x)
    x*=y
     print(x)
```

9

36

```
x-=y
 print(x)
   x/=y
    print(x)
     x%=y
      print(x)
```

3.2

8.0

0.0

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page

Output

```
x=3
y=5
print(x==y)                    False
print(x!=y)                    True
x=5
y=6
print(x>y)                     False
print(x<y)                     True
print(x>=y)                    False
print(x<=y)                    True
```

```
x=True
y=False
print(x and y)                 False
print(x or y)                  True
print(x or y)                  True
print(not x)                   False
print(not y)                   True
```

```
x='24'+'45'
print(x)                       2445
y=24+45
print(Y)                       69
```


Download App

Download Gateway Classes App
Helpline No. : 7455 9612 84

PDF Notes | PYQs Solution | Recorded Lectures | Web Page

```
x = input( "enter a number")
print ( type(x))
y = int(input(" enter a integer"))
  print ( type (y))
z = float( input ("enter a number"))
  print ( type(z))
```

Output

```
enter a number
<class 'str'>
  enter a integer
  <class' int'>
  enter a number
  <class 'float'>
```

```
x=y=z=300
print ("x=", x)
print ("y=", y)
  print ("z=", z)
```

output

```
x=300
y=300
z=300
```

Download Gateway Classes App
Helpline No. : 7455 9612 84
PDF Notes
PYQs Solution
Recorded Lectures
Download App
Web Page